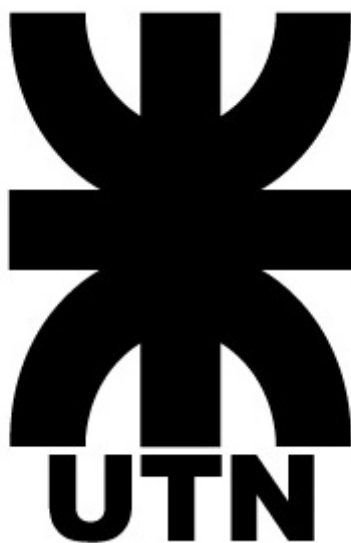




Institución: Universidad Tecnológica Nacional

Carrera: Tecnicatura en programación

Materia: Programación 2

Proyecto: Trabajo practico integrador

Miembros del grupo:

Franco Squillo, Augusto Ingrassia, Matheo Bazan, Elias
Perez, Uriel Vega, Lautaro mejias

Fecha de entrega: 12/06-22/06

Índice

-Marco teórico	3
-Base de datos	3
-POO	
-Clases	3
-Herencia	4
-Sobreescritura de métodos	4
-Interfaces	5
-Relaciones	5
-Programación genérica	
-Clases genéricas	6
-Excepciones	6
-Decisiones técnicas	8
-Dificultades y soluciones	8
-Bibliografía / Webgrafía	9
-Links	9

Marco teórico

-Base de datos

Una base de datos es una colección organizada de información, estructurada de manera que pueda ser fácilmente accedida, gestionada y actualizada. En términos más simples, es como una biblioteca digital donde se almacenan datos de forma ordenada para poder encontrarlos rápidamente cuando los necesites.

Las bases de datos son fundamentales en el mundo digital moderno, permitiendo a las organizaciones almacenar desde información de clientes hasta inventarios completos de productos, historiales médicos, transacciones financieras y mucho más.

Las bases de datos están organizadas en tablas las cuales están divididas en filas y columnas, las filas representan un objeto con sus diferentes atributos y las columnas representa un atributo de un objeto (nombre, edad etc)

Programación Orientada a Objetos (POO)

-Clases

Una clase es una plantilla o modelo utilizado para crear objetos. Define los atributos y métodos que compartirán todas las instancias de dicha clase

Por ejemplo: una clase perro tendrá los atributos de raza y edad y el método de ladrar, en cambio un auto tendrá los atributos de marca y modelo y el método de arrancar.

-Herencia

La herencia es cuando tenemos 2 clases y una de esas clases le “pasa” sus métodos y atributos a la otra clase, la clase que “pasa” sus métodos y atributos se suele llamar **clase padre** y la que los recibe **clase hija**.

La clase hija además de recibir los atributos y métodos de la clase padre también puede tener sus propios métodos y atributos.

Para hacer la herencia se usa la palabra **extends**:

```
public class hija extends padre
```

luego de esto en el constructor de la clase hija se usa la palabra super para llamar al constructor de la clase padre y así poder asignarle valor a los atributos de esta

```
public hijo(String a1,String a2){  
    super(a1)  
    this.a2=a2;  
}
```

-Sobreescritura de métodos

Si en la herencia la clase hija recibe un metodo, esta tiene la posibilidad de sobreescribirlo que básicamente es: usando el mismo método que la clase padre este será modificado y hará algo distinto

Para poder hacer esto se usa la palabra **@Override**

-Método original

```
public void metodo(){  
    System.out.print(“Hola”)  
}
```

-Método con **@Override**

@Override

```
public void metodo(){  
    System.out.print(“Adios”)  
}
```

Para que esto funcione hay algunos requisitos que se tienen que cumplir:

- mismo nombre
- mismos parámetros
- mismo tipo de retorno

-Interfaces

Las interfaces son un contrato de capacidades independiente de la jerarquía. Usar cuando clases sin relación entre sí deben compartir un comportamiento.

Estas pueden tener métodos los cuales las clases que implementen esta interfaz deben modificar con `@Override`.

Una clase puede usar una interfaz con la palabra "implements"

```
public class class implements interface
```

una clase puede llegar a implementar más de una interfaz si es necesario

-Relaciones

En POO, los objetos se relacionan entre sí, y modelamos esas conexiones en el código mediante referencias entre instancias

hay varios tipos de relaciones pero las que usamos en el trabajo son:

- Asociación
- Agregación
- Composición

-Asociación

Un objeto se relaciona con otro ya sea como un atributo de la otra clase (en el caso 1:1) o como parte de un array o un arraylist (en el caso de una relación 1:n o n:m), Los objetos se crean en el main significando que si uno es eliminado el otro sigue existiendo. En el UML se representa como una flecha simple con una línea continua (→).

En el trabajo solo hay una relación de asociación y esta es:

-“DetallePedido” y “Producto” con una cardinalidad N:1

-Agregación

La agregación modela una relación del tipo "tiene un" pero con ciclos de vida independientes. El objeto contenido se crea fuera y luego se pasa al contenedor. Similar a la asociación si es una relación 1:1 se relaciona con un atributo en la otra clase y en el caso de una relación 1:N o N:M es con un array o un arraylist. En el UML se representa con un rombo vacío con una línea continua (-◇)

En el trabajo hay una relación de Agregación y esta es:

-“Categoría” y “Producto” con una cardinalidad 1:N

-Composición

La composición modela una relación con ciclos de vida dependientes. La clase “grande” o el todo recibe por medio de un método los atributos de la clase chica y la crea dentro suyo, lo que hace que si el todo se borra también se borra la otra clase. También se relaciona con un atributo en la relación 1:1 y un array o arraylist en una relación 1:N o N:M. En el UML se representa con un rombo relleno y una línea continua (-◆).

En el trabajo hay una relación de composición y esta es:

“Pedido” y “DetallePedido” siendo una cardinalidad 1:N y siendo “Pedido” el todo

-Programación genérica

-Clases genéricas

Las clases genéricas son clases que nos permiten trabajar con cualquier tipo de dato (int, String, double, etc) reduciendo los errores en el tiempo de ejecución. Mediante parámetros de tipo <T>, los genéricos promueven un código más limpio, flexible y estable.

- Una clase genérica puede utilizarse con cualquier tipo de dato.
- Declarado con un parámetro de tipo (por ejemplo, <T>) después del nombre de la clase.
- Se pueden utilizar varios parámetros de tipo (por ejemplo, <T, U>).
- Los parámetros de tipo deben ser tipos de referencia (no tipos primitivos como int, float, etc.).

-Excepciones

Una excepción es un evento que ocurre durante la ejecución de un programa que interrumpe el flujo normal de las instrucciones del programa más exactamente es un objeto que modela un evento excepcional que no debería haber ocurrido

Ejemplos de situaciones que provocan una excepción:

- Acceso a un elemento de un array fuera de rango
- Leer por teclado un dato de un tipo distinto al esperado
- División por cero

En java todas las excepciones están representadas por clases derivada de la clase “throwable” por ende cuando se produce una excepción se genera un objeto de algún tipo de clase de excepción

El manejo de excepciones Java se gestiona a través de cinco palabras clave: try, catch, throw, throws, y finally.

Bloque try- El bloque de código en el cual puede llegar a surgir una excepción

Bloque catch- Si en el bloque de try se genera una excepción el catch recibe esa excepción y la guarda como un objeto en vez de hacer que el código se detenga, si la excepción es la declarada en el catch se comienza a ejecutar el código dentro del bloque

Bloque finally-Este bloque se ejecutará haya o no una excepción

Ejemplo de código con try-catch-finally

```
try {  
    // Se intenta hacer la división int division = 10 / 0;  
} catch (ArithmeticException a) {  
    // Si la división falla el programa va al bloque catch y se ejecuta el  
    System.out.println  
    System.out.println("Error: división por cero");  
} finally {  
    // Si el programa hizo la división o no, este System.out.print se va a ejecutar  
    igual  
    System.out.println("Saliendo del try");  
}
```

Metodos throwables- Estos son una serie de métodos los cuales nos permiten ver que tipo de excepción ocurrió en el bloque try, normalmente se ponen en el bloque de catch

Métodos

getMessage-Devuelve una descripción de la excepción

fillInStackTrace-Devuelve un objeto Throwable que contiene un seguimiento de pila completo. Este objeto se puede volver a lanzar

toString-Devuelve un objeto String que contiene una descripción completa de la excepción. Este método lo llama println() cuando se imprime un objeto Throwable.

Cláusula throws-La cláusula throws indica qué excepciones puede lanzar un metodo

Ejemplo de declaración de método con throws

```
public int division() throws ArithmeticException
```

Decisiones técnicas

-db

En la carpeta “db” esta la base de datos que vamos a usar y modificar

-lib

En la carpeta “lib” esta el conector con SQL

-main

En la carpeta “main” esta el main el cual es el responsable de ejecutar el código y es el que nos deja interactuar con él por medio de inputs del usuarios

-dao

En la carpeta “dao” es donde se accede a los datos del objeto, también se implementa el CRUD (Create Read Update Delete) y donde se modifica las tablas en la base de datos conectada.

-config

En la carpeta de “config” hay una sola clase donde se configura la conexión a la base de datos

-services

En los servicios estan las validaciones de algunos métodos de los objetos

-entities

En la carpeta “entities” es donde es donde se declaran los atributos y los métodos de las clases que vas a usar y luego que los llamen en el main

-enums

En la carpeta “enums” están los enums que se usan para los atributos de los métodos

Cada tipo de objeto (Categoría / Productos / Usuarios / Pedidos) tienen las 4 mismas opciones (Listar / Crear / Editar / Eliminar) y todas funcionan igual

- listar: Llama a un service que llama a un DAO para que guarde las filas en una lista y luego el main muestra la lista
- Crear: El main pide y recibe los parámetros, luego el service verifica que estén bien y se los pasa al DAO para que los agregue a la tabla.
- Editar: El main pide el id del objeto a modificar para que el DAO lo busque y recibe los nuevos parámetros, luego el service verifica que

estén bien y que el objeto a modificar no esté eliminado y el dao modifica el registro en la tabla

- Eliminar: El main recibe el id del objeto a eliminar para que el DAO lo busque y si lo encuentra cambiar la variable "eliminado" a TRUE

Dificultades y soluciones

-Conexión con la base de datos

El primer problema que tuvimos fue hacer exitosamente la conexión a la base de datos, este problema lo logramos solucionar al pedirle ayuda al profesor Rocco.

El nos dijo que los problemas que teníamos era que en el visual studio code faltaba la extensión para conectarse a la base de datos, que no teníamos hecha la base de datos y problemas de comunicación SCL (Structured Control Language), para solucionar el problema lo que hicimos fue crear una base de datos con el servidor de Xampp y hacer la conexión a esa base de datos y no a la del localhost

Bibliografía / Webgrafía

-Apunte asociaciones POO / profesora Claudia Naveda

-Apunte agregación composición / profesora Claudia Naveda

-Apunte excepciones / profesora Claudia Naveda

-Generic class in java -GeeksforGeeks

<https://www.geeksforgeeks.org/java/generic-class-in-java/>

-Classes and objects -GeeksforGeeks

<https://www.geeksforgeeks.org/java/classes-objects-java/>

inheritance in java -GeeksforGeeks

<https://www.geeksforgeeks.org/java/inheritance-in-java>

Links

-Link del github

https://github.com/francosquillo/TP_Integrador_Programacion2